

pyaino 最小構成から Windows EXE を作成する手順

numpy_package_min → Tsukuyomi.exe

2026 年 9 月 19 日に確定した最終手順

目的	pyaino を NumPy のみで動く最小構成にした numpy_package_min から、単一の Windows 実行ファイル Tsukuyomi.exe を作成する。
方式	PyInstaller 6.22.3 の --onefile / --console を使用。CuPy は --exclude-module cupy で除外する。
同梱データ	Tokenizer 辞書 1 ファイルと、採用したモデル checkpoint 1 ファイルのみ。
最終生成物	D:\Python\pyaino_standalone\numpy_package_min\dist\Tsukuyomi.exe
通常の起動方法	Windows エクスプローラーで dist\Tsukuyomi.exe をダブルクリックする。PowerShell はビルド時や確認時に使用する。
配布先 PC	Python、pyaino、NumPy、PyInstaller の事前インストールは不要。 Tsukuyomi.exe 1 ファイルをコピーし、Windows エクスプローラーからダブルクリックして実行できる。

1. この手順の位置づけ

本書は、2026 年 9 月 19 日の作業で最終的に成功した構成だけを再現手順としてまとめたものである。最終形は、必要な 2 つの pkl ファイルを含め、Python の実行環境まで一つにまとめた単一 EXE である。完成した Tsukuyomi.exe は、配布先 PC に Python、pyaino、NumPy、PyInstaller を別途インストールしなくても実行できる。通常の Windows アプリケーションと同様に、エクスプローラーからダブルクリックして起動する。

要点 「pyaino を最小化する工程」と「最小化済みの numpy_package_min から EXE を作る工程」は分けて考える。本書は後者だけを扱う。

2. 前提環境

9 月 19 日に実際にビルドした環境は次の通りである。

項目	確認された環境
OS	Windows 11
Python	3.12.8
PyInstaller	6.22.3
演算系	NumPy 2.2.6 (CuPy は EXE に含めない)
作業フォルダ	D:\Python\pyaino_standalone\numpy_package_min

PyInstaller は pip で Python 環境へインストールする。9 月 19 日に実際に使用した版は 6.22.3 である。既に同じ版がインストールされている場合、この操作は不要である。

2.1 PyInstaller のインストール

PowerShell を開き、次のコマンドを実行する。9 月 19 日のビルド環境をそのまま再現する場合は、使用した 6.22.3 を指定してインストールする。

```
python -m pip install pyinstaller==6.22.3
```

インストール後、次のコマンドで PyInstaller が使えることを確認する。

```
pyinstaller --version
```

このとき 6.22.3 と表示されれば、9 月 19 日に使用した環境と一致する。

3. 最終的に必要なファイル

numpy_package_min の直下には、少なくとも次の構成を用意する。

```
numpy_package_min\
├── generate_GPT_mix_つくよみちゃん会話_multiturn.py
├── pyaino\
│   └── (最小化済み pyaino 一式)
├── corpus_params\
│   ├── GPT_tokenizer_tsukuyomi_360d12h8l_20260911.pkl
│   └── GPT_params_tsukuyomi_360d12h8l_20260911_itr137088.pkl
```

最終 EXE にデータとして追加するのは、**Tokenizer 辞書**と **checkpoint** の 2 ファイルだけである。学習用データや他の checkpoint は不要である。

4. EXE 化するソースの最終形

PyInstaller の onefile では、実行時に EXE の内容が一時ディレクトリへ展開される。そのため、モデルファイルをカレントディレクトリ基準で読むのではなく、実行中のスクリプト自身の位置を基準に読む。

4.1 基準ディレクトリ

```
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent
```

Tokenizer と checkpoint は次のように BASE_DIR / "corpus_params" から読む。

4.2 データファイルの参照

```
tokenizer.load(
    str(
        BASE_DIR
        / "corpus_params"
        / "GPT_tokenizer_tsukuyomi_360d12h8l_20260911.pkl"
    )
)

cf.load_parameters(
    str(
        BASE_DIR
        / "corpus_params"
        / "GPT_params_tsukuyomi_360d12h8l_20260911_itr137088.pkl"
    ),
    model
)
```

この書き方により、通常の Python 実行では numpy_package_min 直下を、onefile の EXE 実行では PyInstaller の一時展開先 (_MEI...) を基準に、同じコードでファイルを取得できる。

4.3 EXE 実行時だけ不要な内部表示を抑える

配布用 EXE では、pyaino の初期化表示や生成中の fix_configuration 表示を会話画面に出さないようにした。通常の Python 実行時には従来どおり表示できるよう、EXE かどうかで分岐する。

```
import sys
import os
from contextlib import redirect_stdout

IS_EXE = getattr(sys, "frozen", False)

if IS_EXE:
    _stdout = sys.stdout
    _devnull = open(os.devnull, "w", encoding="utf-8")
    sys.stdout = _devnull

# pyaino の import、Tokenizer 読み込み、モデル初期化など
# ...

if IS_EXE:
    sys.stdout = _stdout
    _devnull.close()
```

会話生成時に内部の print を抑える部分は次の形とする。

```
def generate():
    return model.generate(
        seed, len(seed) + 100, stochastic=False,
        skip_ids=skip_ids,
        end_id=end_id, flush=False)

if IS_EXE:
    with open(os.devnull, "w", encoding="utf-8") as f:
        with redirect_stdout(f):
            gen_data = generate()
else:
    gen_data = generate()
```

重要 redirect_stdout を使うので、from contextlib import redirect_stdout を忘れない。最終成功版ではこの形で、会話に必要な「あなた =>」「つくよみ =>」だけを残した。

5. PyInstaller で単一 EXE を作る

PowerShell を開き、まず作業フォルダへ移動する。

```
cd D:\Python\pyaino_standalone\numpy_package_min
```

続いて、9 月 19 日に最終採用した次のコマンドを実行する。

```
pyinstaller --clean --noconfirm --onefile --console --name Tsukuyomi `
--exclude-module cupy `
--add-data "corpus_params\GPT_tokenizer_tsukuyomi_360d12h8l_20260911.pkl:corpus_params" `
--add-data "corpus_params\GPT_params_tsukuyomi_360d12h8l_20260911_itr137088.pkl:corpus_params" `
"generate_GPT_mix_つくよみちゃん会話_multiturn.py"
```

5.1 オプションの意味

オプション

役割

オプション	役割
<code>--clean</code>	以前のビルドキャッシュを消して作り直す。
<code>--noconfirm</code>	既存の出力物があっても確認待ちにせず処理する。
<code>--onefile</code>	配布対象を Tsukuyomi.exe 1 ファイルにまとめる。
<code>--console</code>	会話用のコンソール画面を持つ EXE にする。
<code>--name Tsukuyomi</code>	生成する EXE 名を Tsukuyomi.exe にする。
<code>--exclude-module cupy</code>	NumPy 専用の最小構成なので CuPy を EXE から除外する。
<code>--add-data ...:corpus_params</code>	Tokenizer と checkpoint を EXE に格納し、実行時は corpus_params 配下へ展開する。

このビルドでは Tsukuyomi.spec も自動生成されるが、最終手順では spec の手編集や `--hidden-import` の追加は不要であった。

6. 生成物

ビルドが正常終了すると、作業フォルダの下に build、dist、Tsukuyomi.spec が生成される。配布対象は次の 1 ファイルである。

この Tsukuyomi.exe には、実行に必要な Python ランタイム、pyaino のコード、NumPy、Tokenizer 辞書、モデル checkpoint がまとめて格納されている。したがって、配布先 PC に Python 環境を構築する必要はなく、最終配布物は Tsukuyomi.exe 1 ファイルだけでよい。

```
D:\Python\pyaino_standalone\numpy_package_min\dist\Tsukuyomi.exe
```

- dist\Tsukuyomi.exe : 最終配布物。通常は Windows エクスプローラーからこのファイルをダブルクリックして起動する。
- build\ : ビルド途中の作業領域。配布には不要。
- Tsukuyomi.spec : PyInstaller が自動生成するビルド定義。今回の最終手順では手編集不要。

7. Windows での実行（通常はダブルクリック）

完成した EXE を使うときに PowerShell を開く必要はない。通常の利用では、Windows エクスプローラーで numpy_package_min\dist フォルダを開き、Tsukuyomi.exe をダブルクリックする。

```
dist\Tsukuyomi.exe      (エクスプローラーでダブルクリック)
```

ダブルクリックするとコンソールウィンドウが開き、モデルの読み込み後に次の会話画面になる。--console で作成しているため、input() を使うこのプログラムではコンソールが通常の操作画面となる。

```
つくよみちゃんと会話をします。空打ちで終了します。
```

```
あなた => おはよう
```

```
つくよみ => おはようございます！
```

```
あなた =>
```

入力を空のまま Enter すると会話ループを終了する。最終版では、EXE 実行時の pyaino 初期化ログや MultiHeadSelfAttention fix_configuration の表示を抑え、ダブルクリックで起動したコンソール上に会話に必要な表示だけを残す。

7.1 Python のない PC でそのまま実行できる

PyInstaller の `--onefile` で作成した最終版では、Python インタープリタや必要なライブラリも EXE 内に含まれる。そのため、別の Windows PC に Python、pyaino、NumPy、PyInstaller が入っていないくても、Tsukuyomi.exe だけをコピーしてダブルクリックすれば起動できる。配布を受ける側で `pip install` を行ったり、pyaino のフォルダや corpus_params を別に配置したりする必要はない。すなわち、開発環境で必要だった一式を、利用時には 1 個の EXE として扱えることが今回の EXE 化の最終成果である。

7.2 PowerShell から起動する場合（確認・診断用）

ビルド直後の確認や、エラー表示をその場に残して調べたい場合は、PowerShell から起動してもよい。これは通常の利用方法ではなく、確認・診断用の補助手段である。

```
dist\Tsukuyomi.exe
```

7.3 onefile 特有の一時展開について

onefile の EXE は、起動時に内部ファイルを `_MEI...` という一時ディレクトリへ展開してから実行する。Tokenizer や checkpoint がその一時ディレクトリ内の corpus_params から読まれるのは正常な動作である。 `BASE_DIR = Path(__file__).resolve().parent` と `--add-data` の組合せによって、この動作を意識せずに同じソースを使える。

8. 再作成するときの最短手順

完成済みの numpy_package_min がある状態からの再作成は、次の 5 点だけ確認すればよい。

1. PyInstaller が未導入なら、`python -m pip install pyinstaller==6.22.3` でインストールする。
2. generate_GPT_mix_つくよみちゃん会話_multiturn.py が BASE_DIR 基準で 2 つの pkl を読む最終版であることを確認する。
3. corpus_params に tokenizer と itr137088 checkpoint の 2 ファイルがあることを確認する。
4. numpy_package_min へ `cd` し、5 章の PyInstaller コマンドをそのまま実行する。
5. Windows エクスプローラーで dist\Tsukuyomi.exe をダブルクリックし、会話できることを確認する。
配布時は、この Tsukuyomi.exe 1 ファイルだけを、Python が入っていない Windows PC へコピーして実行できる。

9. 考察

今回のポイントは、pyaino 自体を EXE 化に合わせて別物へ作り替えるのではなく、先に NumPy で自立して動く numpy_package_min を完成させ、その完成物を PyInstaller で包む形にしたことである。これにより、EXE 化のための特殊処理は、データファイルの位置解決と表示制御という外周部にはほぼ限定された。さらに、PyInstaller が Python ランタイムと依存モジュールをまとめるため、完成後は開発用の Python 環境から切り離し、Python を導入していない Windows PC でも EXE 1 ファイルとして利用できる。

10. 結論

結論 最終形では「NumPy 専用に最小化した pyaino + 会話プログラム + tokenizer + checkpoint + Python 実行環境」を PyInstaller の onefile で一つにまとめる。配布物は Tsukuyomi.exe 1 ファイルだけであり、配布先 PC に Python、pyaino、NumPy、PyInstaller をインストールする必要はない。Windows エクスプローラーから Tsukuyomi.exe をダブルクリックして起動するのが通常の使い方である。CuPy、学習用ファイル、他の checkpoint、spec の手編集、hidden-import の追加は必要ない。